ProposalManager.sol

ProposalManager.sol

Purpose:

A lightweight on-chain DAO module to handle proposals and votes for the Sotility ecosystem.
Fully modular — can be used standalone or integrated with Snapshot, Governor contracts, or off-chain oracles.

Core Features:

Proposal submission with metadata

Voting using SOT (Sotility Ownership Token)

Time-based quorum window

Configurable quorum + vote duration

Auto-tally with execution-ready data

--

Plug & Play Use Cases:

Link metadata/IPFS with full proposals

Syncs with frontend UI or DAO dashboards

Can be upgraded later into OZ Governor or Snapshot voting module

--

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title ProposalManager
 * @dev Lightweight DAO module using SOT token for voting.
 */
contract ProposalManager is Ownable {
    enum ProposalStatus { Active, Executed, Rejected }

    struct Proposal {
        string description;
        string ipfsMetadata;
        uint256 startTime;
        uint256 endTime;
        uint256 votesFor;
        uint256 votesAgainst;
        ProposalStatus status;
        mapping(address => bool) hasVoted;
    }

    IERC20 public governanceToken;
    uint256 public quorumThreshold;
    uint256 public voteDuration;

    Proposal[] private proposals;

    event ProposalSubmitted(uint256 proposalId, string description, string ipfsMetadata);
    event Voted(uint256 proposalId, address indexed voter, bool support, uint256 weight);
    event ProposalFinalized(uint256 proposalId, bool passed);

    constructor(address _governanceToken, uint256 _voteDuration, uint256 _quorumThreshold) {
        governanceToken = IERC20(_governanceToken);
        voteDuration = _voteDuration; // in seconds
        quorumThreshold = _quorumThreshold; // in token units (e.g. 1000 * 10^18)
    }

    function submitProposal(string calldata description, string calldata ipfsMetadata) external onlyOwner {
        Proposal storage p = proposals.push();
        p.description = description;
        p.ipfsMetadata = ipfsMetadata;
        p.startTime = block.timestamp;
        p.endTime = block.timestamp + voteDuration;
        p.status = ProposalStatus.Active;
        emit ProposalSubmitted(proposals.length - 1, description, ipfsMetadata);
    }

    function vote(uint256 proposalId, bool support) external {
        require(proposalId < proposals.length, "Invalid proposal");
        Proposal storage p = proposals[proposalId];
        require(block.timestamp < p.endTime, "Voting closed");
        require(!p.hasVoted[msg.sender], "Already voted");

        uint256 weight = governanceToken.balanceOf(msg.sender);
        require(weight > 0, "No SOT balance");

        p.hasVoted[msg.sender] = true;

        if (support) {
            p.votesFor += weight;
        } else {
            p.votesAgainst += weight;
        }

        emit Voted(proposalId, msg.sender, support, weight);
    }

    function finalizeProposal(uint256 proposalId) external {
        require(proposalId < proposals.length, "Invalid proposal");
        Proposal storage p = proposals[proposalId];
        require(block.timestamp >= p.endTime, "Voting still active");
        require(p.status == ProposalStatus.Active, "Already finalized");

        if (p.votesFor + p.votesAgainst >= quorumThreshold && p.votesFor > p.votesAgainst) {
            p.status = ProposalStatus.Executed;
            emit ProposalFinalized(proposalId, true);
        } else {
            p.status = ProposalStatus.Rejected;
            emit ProposalFinalized(proposalId, false);
        }
    }

    function getProposal(uint256 proposalId) external view returns (
        string memory description,
        string memory ipfsMetadata,
        uint256 startTime,
        uint256 endTime,
        uint256 votesFor,
        uint256 votesAgainst,
        ProposalStatus status
    ) {
        Proposal storage p = proposals[proposalId];
        return (
            p.description,
            p.ipfsMetadata,
            p.startTime,
            p.endTime,
            p.votesFor,
            p.votesAgainst,
            p.status
        );
    }

    function totalProposals() external view returns (uint256) {
        return proposals.length;
    }
}
```

Notein
Scan the QR code to download Notein